Name- Sharayu Mohite

Roll NO-B38

PRN NO-2425000616

UNIT1 Fundamental Programming in Java

The Java Programming Environment- JVM, JIT Compiler, A Simple Java Program, Data Types, Variables, Operators, Strings, Input and Output, Control, Flow Declaring Classes, Declaring Member Variables, Defining Methods, Constructor, Passing Information to a Method or a Constructor, Creating and using objects, Controlling Access to Class Members, Static Fields and Methods, this keyword

1. **Explain Procedure Programming language with example of C language. What are advantages and disadvantages of C Programming language**

->

Meaning of Procedure Oriented Programming Language (POP)

Procedure Oriented Programming (POP) is a programming approach where the program is divided into functions or procedures.
Each function performs a specific task.

The program executes step-by-step instructions from top to bottom.

In POP:

- Main focus is on functions (procedures).
- Data and functions are separate.
- The program follows a top-down approach.

Examples of POP Languages

- C
- Pascal
- FORTRAN
- BASIC

Example of Procedure Programming in C

In C language, programs are written using functions such as main(), printf(), or user-defined functions.

Example Program

```
#include <stdio.h>


void display() {
   printf("Welcome to C Programming\n");
}


int main() {
   display();  // function call
   return 0;
}
```

Advantages of C Programming Language

1. Simple and Efficient
   C language has simple syntax and executes programs very fast.
2. Structured Programming
   Programs can be divided into functions which makes coding easier.
3. Portable Language
   C programs can run on different computers with little modification.
4. Rich Library Functions
   Many built-in functions are available.
5. Used for System Programming
   Operating systems, compilers, and embedded systems are written in C.
6. Memory Management
   C allows direct memory access using pointers.

Disadvantages of C Programming Language

1. No Object Oriented Features
   C does not support classes, objects, inheritance, etc.
2. Manual Memory Management
   Programmer must handle memory allocation manually.
3. No Data Security
   Data is not well protected compared to OOP languages.
4. Complex Pointer Concept
   Pointers can be difficult for beginners.
5. No Exception Handling
   C does not provide built-in error handling mechanisms.

Procedure Oriented Programming focuses on functions and step-by-step execution.
C language is a popular POP language used for system programming, embedded systems, and application development.

2. What is Python? Explain its advantages and disadvantages. Write a simple Python program to implement a Student Result System that calculates total and average marks of a student

What is Python?

Python is a high-level, interpreted programming language that is easy to learn and use.

It was created by Guido van Rossum in 1991.

Python is widely used for:

- Web development
- Artificial Intelligence
- Machine Learning
- Data Science
- Automation

Python uses simple and readable syntax, which makes it beginner-friendly.

Advantages of Python

1. Easy to Learn
   Python syntax is simple and similar to English.
2. Less Coding Required
   Programs are shorter compared to other languages.
3. Large Library Support
   Python provides many libraries for AI, data science, and web development.
4. Platform Independent
   Python programs can run on Windows, Linux, and Mac.
5. Supports Multiple Programming Styles
   It supports procedural, object-oriented, and functional programming.
6. Open Source Language
   Python is free to use.

Disadvantages of Python

1. Slow Execution Speed
   Python is slower than compiled languages like C or C++.
2. High Memory Usage
   Python programs use more memory.
3. Not Suitable for Mobile Development
   Python is rarely used for mobile applications.
4. Weak Database Access
   Database layers are less powerful compared to some other languages.

This program calculates total marks and average marks of a student.

Python Program

```
name = input("Enter student name: ")

m1 = int(input("Enter marks of Subject 1: "))
m2 = int(input("Enter marks of Subject 2: "))
m3 = int(input("Enter marks of Subject 3: "))

total = m1 + m2 + m3
average = total / 3
```

```
print("Student Name:", name)
print("Total Marks:", total)
print("Average Marks:", average)
```

Python is a powerful and easy-to-learn programming language widely used in modern technologies such as AI, data science, and automation.
Its simple syntax makes it ideal for beginners and large-scale applications.

3. **Explain the concept of Object-Oriented Programming (OOP). Discuss the main principles of OOP and describe the important features of Java**

1. Concept of Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming approach where a program is organized using objects and classes.

An object represents a real-world entity such as a student, car, or bank account.
A class is a blueprint or template used to create objects.

In OOP, data and functions are combined into a single unit called an object.

This concept helps programmers create modular, reusable, and secure programs.

Example

If we create a Student class, it may contain:

- Data: name, roll number, marks
- Methods: display(), calculateResult()

Example (Java):

```
class Student {
  String name;
  int marks;

  void display() {
    System.out.println(name + " " + marks);
  }
}
```

Here:

- Data is **hidden using private access modifier**
- Methods control access to the data.

Inheritance

Inheritance means **one class can acquire properties and methods of another class**.

It promotes **code reuse**.

Example:

```
class Animal {
  void eat() {
    System.out.println("Animal eats");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Dog barks");
  }
}
```

Here:

- Dog inherits properties from Animal.

Polymorphism

Polymorphism means **one function can perform different tasks depending on the situation**.

There are two types:

1. **Compile-time polymorphism (Method Overloading)**
2. **Runtime polymorphism (Method Overriding)**

Example:

```
class Add {
  int sum(int a, int b) {
    return a + b;
  }
```

```
  int sum(int a, int b, int c) {
    return a + b + c;
  }
}
```

Here the same method name sum() works differently.

Abstraction

Abstraction means **hiding internal details and showing only important information**.

It helps reduce program complexity.

Example:

- When driving a car, we use **steering and pedals**, but we do not see the internal engine process.

In Java, abstraction is achieved using:

- **Abstract classes**
- **Interfaces**

Important Features of Java

Java is a popular object-oriented programming language developed by **Sun Microsystems** (now owned by **Oracle Corporation**).

Important features of Java are:

3.1 Simple

Java syntax is easy to learn and similar to C and C++.

3.2 Object-Oriented

Java follows OOP concepts such as:

- Class
- Object
- Inheritance
- Polymorphism
- Encapsulation

- Abstraction

3.3 Platform Independent

Java follows the concept "**Write Once, Run Anywhere (WORA)**".

Java programs run on **Java Virtual Machine (JVM)**.

3.4 Secure

Java provides security features such as:

- Bytecode verification
- No pointer access
- Secure memory management

3.5 Robust

Java has strong **memory management and exception handling**, which makes programs reliable.

3.6 Multithreading

Java supports **multiple threads**, allowing programs to perform several tasks at the same time.

Example:

- Downloading files while playing music.

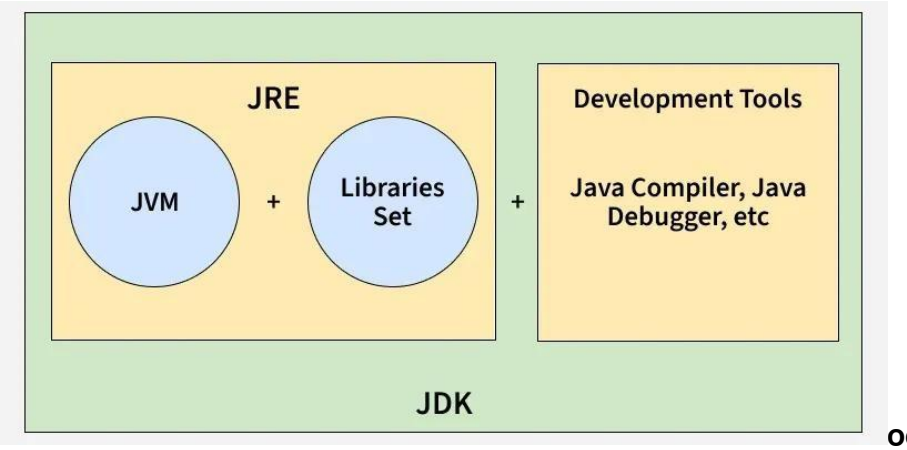
3.7 Portable

Java programs can run on different operating systems without modification.

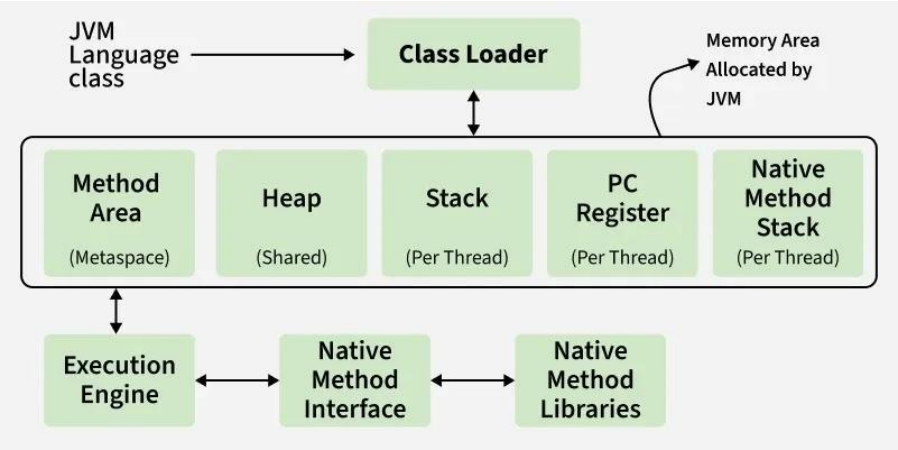
3.8 High Performance

Java uses **Just-In-Time (JIT) compiler** which improves execution speed.

4. Explain JDK, JRE and JVM with the block Diagram. List the steps and command used to compile and run Java source C



ode



JDK (Java Development Kit) provides tools and libraries to develop Java applications, working with JRE and JVM. JRE (Java Runtime Environment) offers the libraries and JVM needed to run Java programs. JVM (Java Virtual Machine) executes the compiled Java bytecode on the system.

JDK (Java Development Kit)

JDK is a software development kit used to build Java applications. It contains the JRE and a set of development tools.

- Includes compiler (javac), debugger, and utilities like jar and javadoc.
- Provides the JRE, so it also allows running Java programs.
- Required by developers to write, compile, and debug code.

Components of JDK:

- JRE (JVM + libraries)
- Development tools (compiler, jar, javadoc, debugger)

Note:

- JDK is only for development (it is not needed for running Java programs)
- JDK is platform-dependent (different version for windows, Linux, macOS)

Working of JDK:

- **Source Code (.java):** Developer writes a Java program.
- **Compilation:** The JDK's compiler (javac) converts the code into bytecode stored in .class files.
- **Execution:** The JVM executes the bytecode, translating it into native instructions.

JRE (Java Runtime Environment)

JRE provides an environment to run Java programs but does not include development tools. It is intended for end-users who only need to execute applications.

- Contains the JVM and standard class libraries.
- Provides all runtime requirements for Java applications.
- Does not support compilation or debugging.

Note:

- JRE is only for running applications, not for developing them.
- It is platform-dependent (different builds for different OS).

Working of JRE:

1. **Class Loading:** Loads compiled .class files into memory.
2. **Bytecode Verification:** Ensures security and validity of bytecode.
3. **Execution:** Uses the JVM (interpreter + JIT compiler) to execute instructions and make system calls.

JVM (Java Virtual Machine)

JVM is the core execution engine of Java. It is responsible for converting bytecode into machine-specific instructions.

- Part of both JDK and JRE.
- Performs memory management and garbage collection.
- Provides portability by executing the same bytecode on different platforms.

Note:

- JVM implementations are platform-dependent.

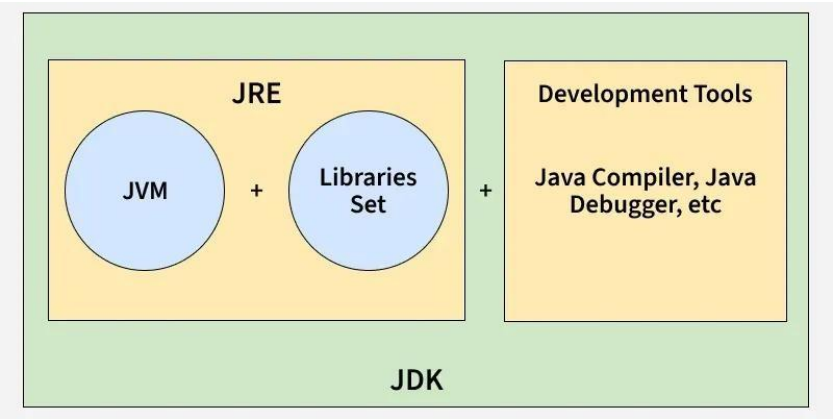
- Bytecode is platform-independent and can run on any JVM.
- Modern JVMs rely heavily on Just-In-Time (JIT) compilation for performance.

Working of JVM:

JVM working

1. **Loading:** Class loader loads bytecode into memory.
2. **Linking:** Performs verification, preparation, and resolution.
3. **Initialization:** Executes class constructors and static initializers.
4. **Execution:** Interprets or compiles bytecode into native code.

5. **Explain Java Virtual Machine and its various component with neat diagram**



JVM (Java Virtual Machine)

JVM is the core execution engine of Java. It is responsible for converting bytecode into machine-specific instructions.

- Part of both JDK and JRE.
- Performs memory management and garbage collection.
- Provides portability by executing the same bytecode on different platforms.

Note:

- JVM implementations are platform-dependent.
- Bytecode is platform-independent and can run on any JVM.
- Modern JVMs rely heavily on Just-In-Time (JIT) compilation for performance.

Working of JVM:

JVM working

1. **Loading:** Class loader loads bytecode into memory.
2. **Linking:** Performs verification, preparation, and resolution.
3. **Initialization:** Executes class constructors and static initializers.
4. **Execution:** Interprets or compiles bytecode into native code.

6. Describe Primitive Datatypes available in Java with help of purpose, Syntax and Example

Primitive datatypes are the **basic built-in data types** provided by Java to store simple values such as numbers, characters, and logical values. They are not objects and directly store the actual value in memory. Java provides **8 primitive data types**.

1. byte
Purpose: Used to store very small integer values and to save memory. Range is –128 to 127.
Syntax: byte variableName;
Example:

```
byte age = 20;
System.out.println(age);
```

2. short
Purpose: Used to store small integer numbers when memory saving is required. Range is –32,768 to 32,767.
Syntax: short variableName;
Example:

```
short marks = 300;
System.out.println(marks);
```

3. int
Purpose: Used to store integer numbers. It is the most commonly used datatype for numeric values.
Syntax: int variableName;
Example:

```
int salary = 50000;
System.out.println(salary);
```

4. long
Purpose: Used to store very large integer values when int is not sufficient.
Syntax: long variableName;
Example:

```
long population = 9000000000L;
System.out.println(population);
```

5. float
Purpose: Used to store decimal numbers with single precision.
Syntax: float variableName;
Example:

```
float price = 45.75f;
System.out.println(price);
```

6. double
Purpose: Used to store large decimal numbers with double precision. It is commonly used for scientific calculations.
Syntax: double variableName;
Example:

```
double percentage = 89.56;
System.out.println(percentage);
```

7. char
Purpose: Used to store a single character such as a letter, digit, or symbol.
Syntax: char variableName;
Example:

```
char grade = 'A';
System.out.println(grade);
```

8. boolean
Purpose: Used to store logical values (true or false). Mainly used in conditions and decision making.
Syntax: boolean variableName;
Example:

```
boolean result = true;
System.out.println(result);
```

Primitive datatypes in Java are used to store simple values efficiently. They form the

basic building blocks of Java programs and help manage memory and data operations effectively.

7. Describe Arithmetic and Assignment operators with help of Symbol, Name, Description and apply the operators on x and y variable for the example demonstration

Arithmetic and Assignment Operators in Java

Operators are special symbols used to **perform operations on variables and values**. Arithmetic operators perform **mathematical calculations**, while assignment operators are used to **assign values to variables**.

Assume the following values for demonstration:

int x = 10;
int y = 5;

1. Arithmetic Operators

Arithmetic operators are used to perform **basic mathematical operations** such as addition, subtraction, multiplication, and division.

Symbol	Name	Description	Example using x and y
+	Addition	Adds two values	x + y = 10 + 5 = 15
-	Subtraction	Subtracts second value from first	x - y = 10 - 5 = 5
*	Multiplication	Multiplies two values	x * y = 10 × 5 = 50
/	Division	Divides first value by second	x / y = 10 / 5 = 2
%	Modulus	Gives remainder after division	x % y = 10 % 5 = 0

```
class ArithmeticExample {
  public static void main(String args[]) {
    int x = 10;
    int y = 5;

    System.out.println("Addition: " + (x + y));
```

```
    System.out.println("Subtraction: " + (x - y));
    System.out.println("Multiplication: " + (x * y));
    System.out.println("Division: " + (x / y));
    System.out.println("Modulus: " + (x % y));
  }
}
```

Assignment operators are used to **assign values to variables.**
They can also combine arithmetic operations with assignment.

Symbol	Name	Description	Example using x and y
=	Simple Assignment	Assigns value to variable	x = y → x becomes 5
+=	Add and Assign	Adds value and assigns result	x += y → x = x + y = 15
-=	Subtract and Assign	Subtracts value and assigns result	x -= y → x = 5
*=	Multiply and Assign	Multiplies value and assigns result	x *= y → x = 50
/=	Divide and Assign	Divides value and assigns result	x /= y → x = 2
%=	Modulus and Assign	Finds remainder and assigns result	x %= y → x = 0

```
class AssignmentExample {
  public static void main(String args[]) {
    int x = 10;
    int y = 5;

    x += y;
    System.out.println("x += y: " + x);

    x -= y;
    System.out.println("x -= y: " + x);
```

```
        x *= y;
        System.out.println("x *= y: " + x);

        x /= y;
        System.out.println("x /= y: " + x);

        x %= y;
        System.out.println("x %= y: " + x);
    }
}
```

8. Explain the Java Comments, Source File, Class, Method Declaration rules with Sample Java Source Code

Rules of Java Comments, Source File, Class and Method Declaration in Java

Java programs follow certain rules for writing comments, creating source files, declaring classes, and defining methods. These rules help in writing **clear, organized, and error-free programs**.

1. Java Comments
Comments are notes written in a program to explain the code. They are ignored by the compiler and do not affect program execution. Comments help programmers understand the logic of the program.

Types of Java Comments:
Single-line comment – Used for a single line explanation.
Symbol: //
Example:

```
// This program prints a message
```

Multi-line comment – Used for writing comments across multiple lines.
Symbol: /* */
Example:

```
/* This is a multi-line comment
  used to explain the program */
```

Documentation comment – Used to generate documentation using Javadoc tool.
Symbol: /** */
Example:

```
/**
 * This class displays a message
 */
```

2. Java Source File
A source file is the file where Java program code is written. It contains classes, methods, variables, and statements.

Rules of Java Source File:
The file must have the extension **.java**.
The file name must be the **same as the public class name**.
A source file may contain **multiple classes**, but only one class can be public.
The program execution always starts from the **main() method**.

Example:
If the class name is Hello, the file name must be:

Hello.java

3. Java Class Declaration Rules
A class is a blueprint used to create objects. It contains variables and methods that define the behavior of objects.

Rules for declaring a class:
The keyword **class** is used to declare a class.
The class name should start with a **capital letter** by convention.
The class name should match the **source file name** if it is public.
A class body is written inside **curly braces { }**.

Syntax:

```
class ClassName {
  // variables
  // methods
}
```

Example:

```
class Student {
  int id;
  String name;
}
```

4. Java Method Declaration Rules

A method is a block of code that performs a specific task. Methods improve code reuse and program organization.

Rules for declaring a method:
A method must have a **return type**.
A method must have a **method name**.
Parentheses () are used to pass parameters.
The method body is written inside **curly braces { }**.

Syntax:

```
returnType methodName(parameters) {
  // method body
}
```

Example:

```
void display() {
  System.out.println("Welcome to Java");
}
```

5. Sample Java Source Code

```
// Simple Java Program Example

class HelloWorld {

  // main method: program execution starts here
  public static void main(String args[]) {

    /* printing message on screen */
    System.out.println("Hello Java");
  }
}
```

Explanation:
// is used for single-line comments.
/* */ is used for multi-line comments.

HelloWorld is the class name.
main() is the method where program execution begins.
System.out.println() prints output on the screen.

9. What is Scanner Class? Explain various methods available in Scanner class. Write a sample program to take String, integer, Float and double variables from command prompt.

1. What is Scanner Class

The **Scanner class** is used in Java to take **input from the user at runtime**. It is part of the **java.util package**. Scanner reads different types of data such as **String, integer, float, and double** from the keyboard (command prompt).

To use Scanner in a program, we must import it first.

Syntax to import Scanner:

import java.util.Scanner;

Scanner reads input using **methods like nextInt(), nextFloat(), nextDouble(), and nextLine()**.

General syntax to create Scanner object:

Scanner sc = new Scanner(System.in);

Here:
Scanner → class name
sc → object name
System.in → standard input stream (keyboard)

---

2. Various Methods Available in Scanner Class

nextInt()
Purpose: Reads an **integer value** from user input.
Example:

int num = sc.nextInt();

nextFloat()
Purpose: Reads a **float (decimal) value** from user input.
Example:

```
float price = sc.nextFloat();
```

nextDouble()
Purpose: Reads a **double precision decimal number**.
Example:

```
double percentage = sc.nextDouble();
```

next()
Purpose: Reads **a single word string** from input.
Example:

```
String name = sc.next();
```

nextLine()
Purpose: Reads **an entire line of text including spaces**.
Example:

```
String sentence = sc.nextLine();
```

nextBoolean()
Purpose: Reads **true or false values**.
Example:

```
boolean result = sc.nextBoolean();
```

---

3. Sample Program to Take String, Integer, Float and Double Input

```
import java.util.Scanner;

class ScannerExample {
  public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter your name: ");
    String name = sc.nextLine();

    System.out.print("Enter your age: ");
    int age = sc.nextInt();

    System.out.print("Enter your height (float): ");
    float height = sc.nextFloat();

    System.out.print("Enter your percentage (double): ");
```

```
        double percentage = sc.nextDouble();

        System.out.println("\n--- Student Details ---");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Height: " + height);
        System.out.println("Percentage: " + percentage);
    }
}
```

10. What is Class and Object in Java? Explain Data members and Methods of Class with the help of diagram and example.

1. What is a Class

A **class** is a blueprint or template used to create objects. It defines the **properties (data)** and **behaviors (methods)** that objects of that class will have. In Java, a class groups related variables and functions together so that programs are organized and reusable.

Example:
If we create a **Student class**, it may contain:

- Data: name, roll number, marks
- Methods: display(), calculateResult()

Syntax of Class

```
class ClassName {
    // data members
   // methods
}
```

2. What is an Object

An **object** is an instance of a class. It represents a real-world entity and uses the variables and methods defined in the class.

Example:
If Student is a class, then s1 and s2 can be objects of that class.

Syntax to create object

ClassName objectName = new ClassName();

Example:

Student s1 = new Student();

Here
Student → class name
s1 → object
new → keyword used to create object

3. Data Members of a Class

Data members are variables declared inside a class. They store the **data or attributes of an object**.

Example attributes of Student:

- name
- rollNo
- marks

Example:

```
class Student {
  int rollNo;
  String name;
  int marks;
}
```

Here rollNo, name, and marks are **data members**.

Purpose of Data Members:

- Store information about objects
- Represent properties of an object
- Used by methods to perform operations

4. Methods of a Class

A **method** is a function defined inside a class that performs a specific task. Methods operate on the data members of the class.

Example:

```
void display() {
  System.out.println("Student details");
}
```

Purpose of Methods:

- Perform operations on data
- Define behavior of objects
- Improve code reuse and modularity

5. Diagram of Class, Data Members and Methods

```
    Class : Student
  -----------------
  | Data Members        |
  | name                |
  | rollNo              |
  | marks               |
  |                     |
  | Methods             |
  | display()           |
  | calculateResult()   |
  -----------------

        |
        | create object
        v

    Object : s1
```

The object **s1** will contain its own values for the data members and can use the methods of the class.

6. Example Program

```
class Student {

  int rollNo;
```

```
    String name;

    void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }

    public static void main(String args[]) {

        Student s1 = new Student();

        s1.rollNo = 101;
        s1.name = "Rahul";

        s1.display();
    }
}
```

Explanation:

- Student is a **class**
- rollNo and name are **data members**
- display() is a **method**
- s1 is an **object of the class**

11. Write a short note

Operators – new operator, instanceof operator, dot(.) Operator

If, else, if-else with syntax and example

A. Operators in Java

1. new Operator

The **new operator** is used to **create objects of a class**. When the new operator is used, memory is allocated for the object and the constructor of the class is called.

Syntax

ClassName objectName = new ClassName();

Example

```
class Student {
  int id;
}

class Test {
  public static void main(String args[]) {
    Student s1 = new Student();
  }
}
```

Explanation:
new Student() creates an object of the Student class and allocates memory.

2. instanceof Operator

The **instanceof operator** is used to check whether an object belongs to a particular class or not. It returns a **boolean value (true or false)**.

Syntax

objectName instanceof ClassName

Example

```
class Test {
  public static void main(String args[]) {
    String name = "Java";

    boolean result = name instanceof String;
    System.out.println(result);
  }
}
```

Output:

true

Explanation:
It checks whether the object name belongs to the String class.

3. Dot (.) Operator

The **dot operator** is used to **access variables and methods of a class using an object**.

Syntax

objectName.variableName
objectName.methodName()

Example

```
class Student {
  int rollNo = 10;

  void display() {
    System.out.println("Hello Student");
  }
}

class Test {
  public static void main(String args[]) {
    Student s1 = new Student();

    System.out.println(s1.rollNo);
    s1.display();
  }
}
```

Explanation:
The dot operator allows access to **data members and methods** of the object.

B. If, Else, If-Else Statements

Conditional statements are used to **make decisions in a program based on conditions**.

1. if Statement

The **if statement** executes a block of code only when the condition is **true**.

Syntax

```
if(condition) {
  // statements
}
```

Example

```
int marks = 60;

if(marks >= 50) {
  System.out.println("Pass");
}
```

2. if-else Statement

The **if-else statement** is used when we want to execute **one block if condition is true and another block if it is false**.

Syntax

```
if(condition) {
  // true statements
}
else {
  // false statements
}
```

Example

```
int number = 10;

if(number % 2 == 0) {
  System.out.println("Even Number");
}
else {
  System.out.println("Odd Number");
}
```

3. else-if Ladder

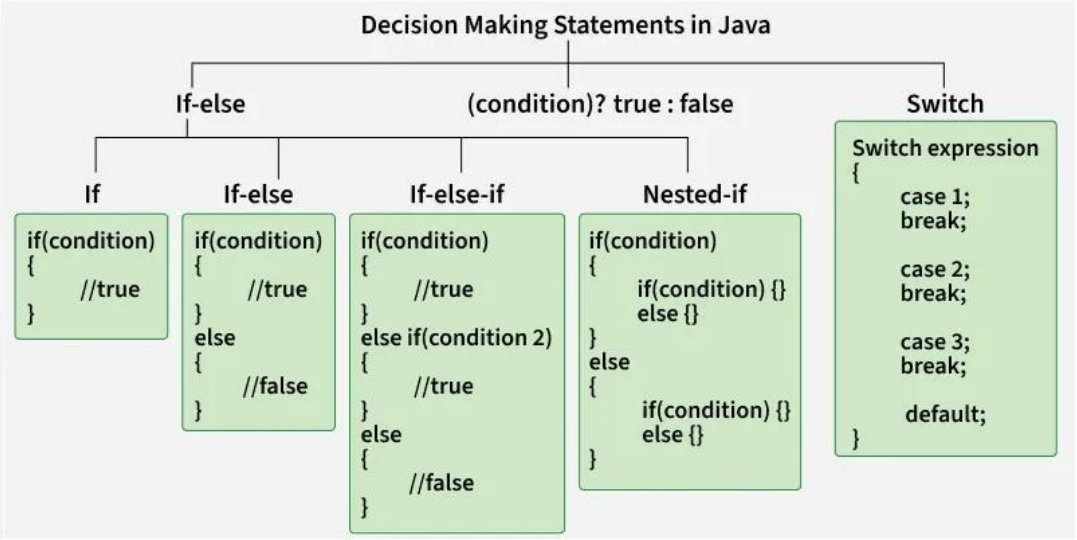
The **else-if ladder** is used when multiple conditions need to be checked.

Syntax

```
if(condition1) {
    // statement
}
else if(condition2) {
   // statement
}
else {
   // statement
}
```

Example

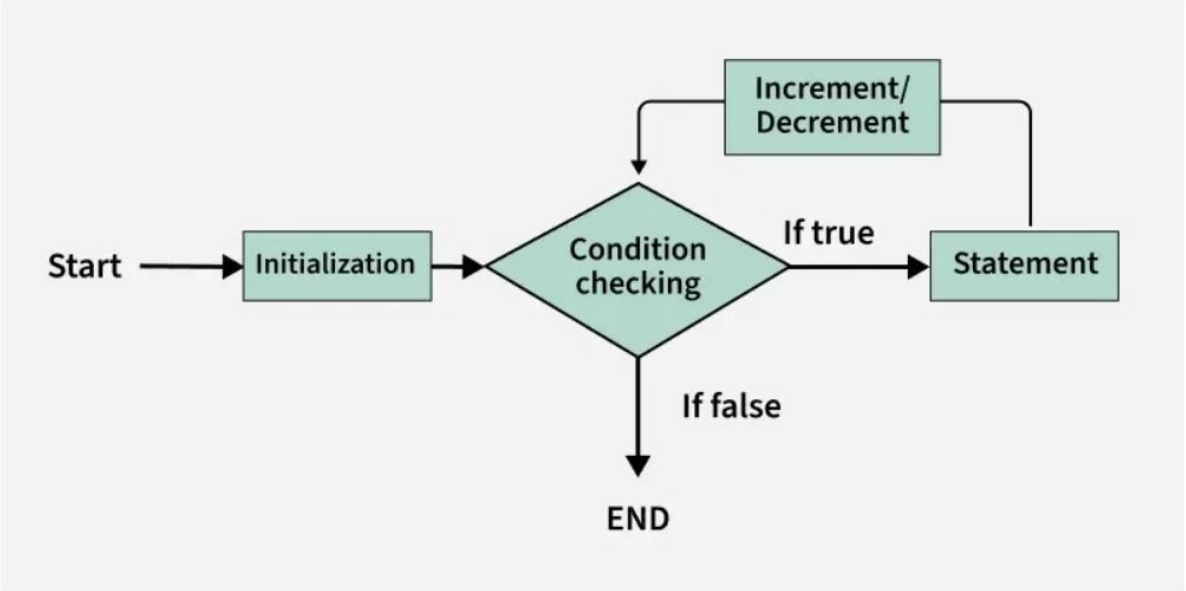
```
int marks = 75;

if(marks >= 80) {
   System.out.println("Grade A");
}
else if(marks >= 60) {
   System.out.println("Grade B");
}
else {
   System.out.println("Grade C");
}
```



12. Design flow charts for 1. for loop 2. While Loop 3. do-while loop 4. if-else condition 5. Switch Statement

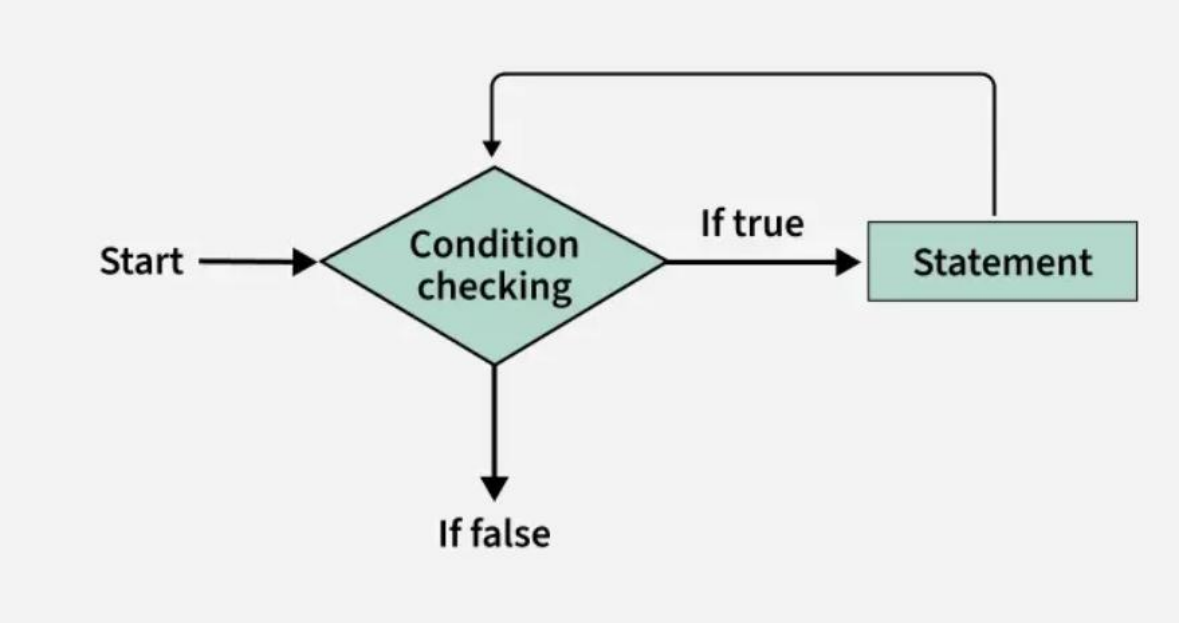
1. Flowchart for For Loop

The image below demonstrates the flow chart of a for loop:



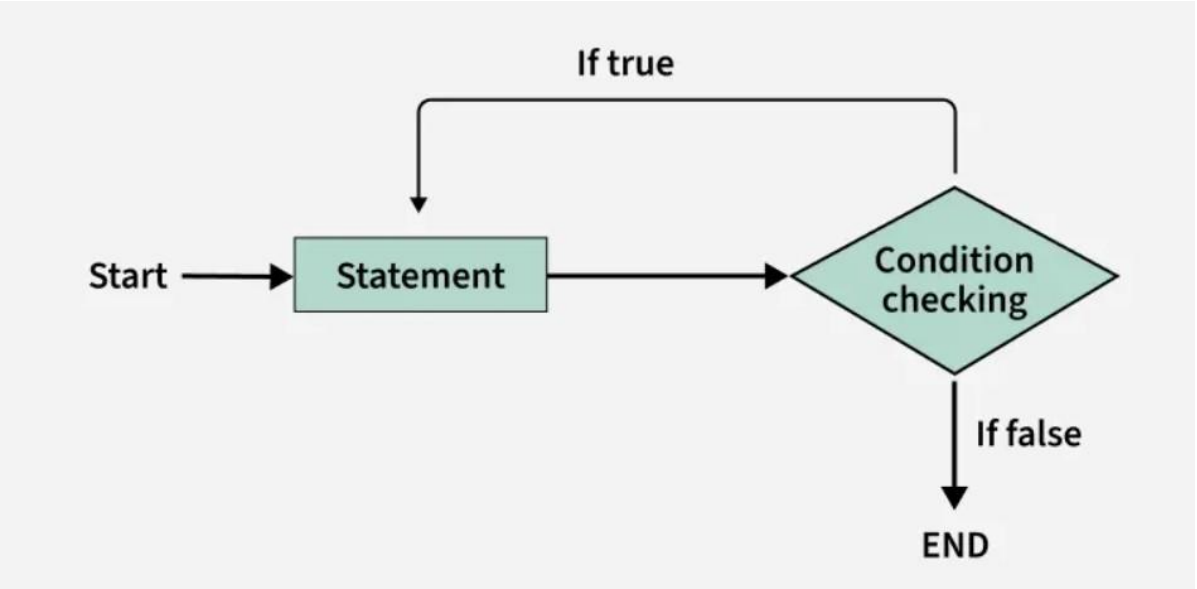
Explanation: The **for loop** first initializes the variable, checks the condition, executes the statements if true, updates the variable, and repeats until the condition becomes false.

2. Flowchart for While Loop



Explanation: The **while loop** checks the condition first. If it is true the loop body executes, otherwise the loop terminates.

3. Flowchart for Do-While Loop



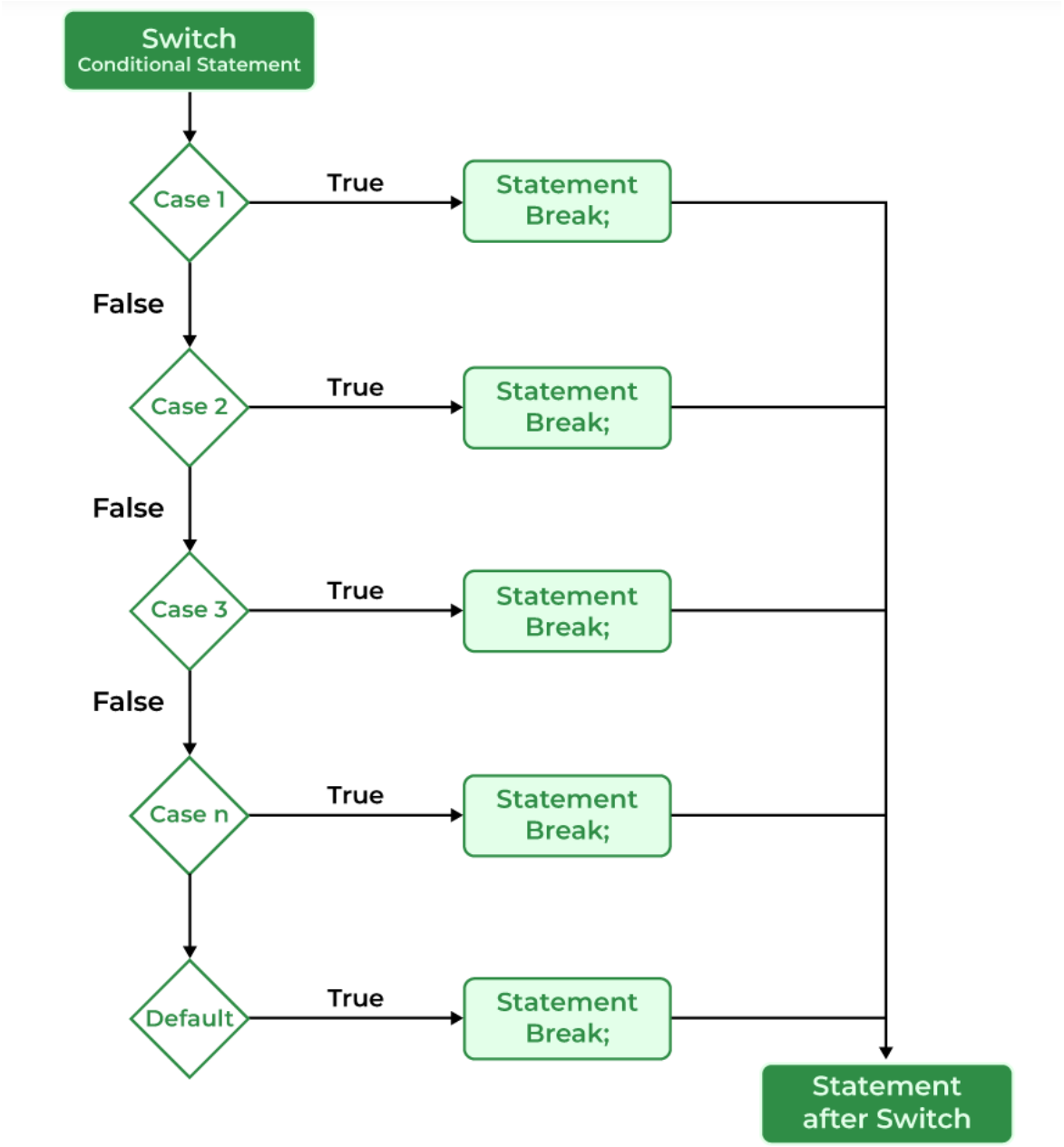
Explanation: In the **do-while loop**, the loop body executes at least once before checking the condition.

4. Flowchart for If-Else Condition

```
Start
 |
Check condition
 |True       |False
 v           v
Execute IF    Execute ELSE
statements     statements
    \       /
     \     /
       End
```

Explanation: The **if-else statement** checks a condition. If it is true, the IF block executes; otherwise the ELSE block executes.

5. Flowchart for Switch Statement



Explanation: The **switch statement** compares the expression with different cases. When a matching case is found, the corresponding statements execute; otherwise the default case executes.

13. Why static keyword is used in java? Explain Static variable and static method implementation with help of code examples and diagram as suitable.

The **static keyword** in Java is used to create variables and methods that **belong to the class rather than to objects**. A static member is **shared by all objects of the class** and memory for it is allocated only once. Static members can be accessed **without**

creating an object of the class. Static is commonly used for **memory management, common properties, and utility methods**.

Static Variable

A **static variable** is also called a **class variable** because it is shared by all objects of the class. Only one copy of the variable exists in memory and every object uses that same copy. Static variables are generally used when the value is **common for all objects**, such as a college name shared by all students.

Syntax

static datatype variableName;

Example Program

```
class Student {
  int rollNo;
  String name;
  static String college = "ABC College";

  void display() {
    System.out.println(rollNo + " " + name + " " + college);
  }

  public static void main(String args[]) {
    Student s1 = new Student();
    s1.rollNo = 1;
    s1.name = "Rahul";

    Student s2 = new Student();
    s2.rollNo = 2;
    s2.name = "Neha";

    s1.display();
    s2.display();
  }
}
```

Explanation: Both objects **s1 and s2 share the same static variable college**, so the value is stored only once in memory.

Diagram of Static Variable Sharing

```
      Class : Student
  ----------------------
```

```
| static college = "ABC College"|
---------------------------------
      /            \
     /              \
 Object s1          Object s2
rollNo=1           rollNo=2
name=Rahul         name=Neha
  |                |
  ---- share same static variable ----
```

Static Method

A **static method** belongs to the class instead of objects. It can be called directly using the **class name** without creating an object. Static methods can access **only static variables and other static methods**.

Syntax

```
static returnType methodName() {
  // statements
}
```

Example Program

```
class Test {

  static void message() {
    System.out.println("Welcome to Java Programming");
  }

  public static void main(String args[]) {
    Test.message();
  }
}
```

Explanation: The method message() is static, so it is called using the class name **Test.message()** without creating an object.

Diagram of Static Method Access

```
      Class : Test
  --------------------
  | static message() method  |
  --------------------
        |
        | call using
```

```
        v
    Test.message()
        |
     Output
 "Welcome to Java Programming"
```

Advantages of Static Keyword

Used when a value is **common for all objects**. Saves **memory because only one copy exists**. Static methods allow calling functions **without creating objects**. Frequently used for **utility functions and constants**.

14. Explain String pool Concept in Java using diagram. Explain 4 String class methods with help of syntax, input and output code syntax

The **String Pool** (also called **String Constant Pool**) is a special memory area inside the **heap** used to store **string literals**. When a string is created using double quotes, Java checks the pool first. If the same string already exists, Java **reuses the existing object instead of creating a new one**. This improves **memory efficiency and performance** because duplicate strings are not created.

Example concept:

```
String s1 = "Java";
String s2 = "Java";
String s3 = new String("Java");
```

Here s1 and s2 refer to the **same object in the string pool**, while s3 creates a **new object in heap memory**.

Diagram of String Pool

```
        Heap Memory
    -------------------
    |   String Pool     |
    |                   |
    | "Java" <---- s1   |
    |        <---- s2   |
    |                   |
    -------------------
          |
          |
        Heap Object
```

```
"Java"
 ^
 |
 s3
```

Explanation: s1 and s2 point to the same pooled string "Java". s3 creates a separate object because it uses the new keyword.

String Class Methods

The **String class** provides many methods to manipulate strings. Below are four commonly used methods with syntax, input, and output examples.

1. length() Method

Purpose: Returns the number of characters in a string.
Syntax:

```
stringName.length();
```

Example:

```
class Test {
  public static void main(String args[]) {
    String str = "Java";
    System.out.println(str.length());
  }
}
```

Input: Java
Output:

```
4
```

2. toUpperCase() Method

Purpose: Converts all characters of a string to uppercase.
Syntax:

```
stringName.toUpperCase();
```

Example:

```
class Test {
  public static void main(String args[]) {
    String str = "java";
    System.out.println(str.toUpperCase());
  }
}
```

Input: java
Output:

JAVA

3. charAt() Method

Purpose: Returns the character at a specified index position.
Syntax:

```
stringName.charAt(index);
```

Example:

```
class Test {
  public static void main(String args[]) {
    String str = "Java";
    System.out.println(str.charAt(2));
  }
}
```

Input: Java
Output:

v

4. substring() Method

Purpose: Returns a part of the string from the specified index.
Syntax:

```
stringName.substring(startIndex, endIndex);
```

Example:

```
class Test {
  public static void main(String args[]) {
    String str = "Programming";
    System.out.println(str.substring(0,6));
  }
}
```

Input: Programming
Output:

Progra

15. Explain constructor and its types in detail. Write a java program to demonstrate constructor overloading

A **constructor** is a special method in Java that is used to **initialize objects of a class**. It is automatically called when an object of the class is created. The constructor initializes the values of data members.

Important Characteristics of Constructor

Constructor name must be the **same as the class name**. A constructor **does not have any return type** (not even void). It is **automatically executed when an object is created** using the new keyword. Constructors are mainly used to **initialize object variables.**

Syntax

```
class ClassName {
  ClassName() {
    // initialization code
  }
}
```

Example:

```
class Test {
  Test() {
    System.out.println("Constructor Called");
  }

  public static void main(String args[]) {
    Test t = new Test();
  }
}
```

Output

Constructor Called

Types of Constructors in Java

1. **Default Constructor**

A **default constructor** is a constructor that does not take any parameters. If a programmer does not define any constructor, Java automatically provides a default constructor.

Example:

```
class Student {
  int id;
  String name;

  Student() {
    id = 1;
    name = "Rahul";
  }

  void display() {
    System.out.println(id + " " + name);
  }

  public static void main(String args[]) {
    Student s1 = new Student();
    s1.display();
  }
}
```

2. Parameterized Constructor

A **parameterized constructor** accepts arguments so that different objects can be initialized with different values.

Example:

```
class Student {
  int id;
  String name;

  Student(int i, String n) {
    id = i;
    name = n;
  }

  void display() {
    System.out.println(id + " " + name);
  }

  public static void main(String args[]) {
    Student s1 = new Student(101, "Rahul");
```

```
    s1.display();
  }
}
```

Diagram of Constructor Working

```
      Class Student
   -----------------
   | id              |
   | name            |
   | Student() constructor |
   -----------------
          |
          | object creation
          v
        Student s1
      Constructor runs
     id and name initialized
```

Constructor Overloading

Constructor overloading means defining **multiple constructors in the same class with different parameters**. Each constructor performs different initialization based on the arguments passed.

Java Program to Demonstrate Constructor Overloading

```
class Student {

  int id;
  String name;
  int age;

  Student() {
    id = 0;
    name = "Unknown";
    age = 0;
  }

  Student(int i, String n) {
    id = i;
    name = n;
  }
```

```
  Student(int i, String n, int a) {
    id = i;
    name = n;
    age = a;
  }

  void display() {
    System.out.println(id + " " + name + " " + age);
  }

  public static void main(String args[]) {

    Student s1 = new Student();
    Student s2 = new Student(101, "Rahul");
    Student s3 = new Student(102, "Neha", 20);

    s1.display();
    s2.display();
    s3.display();
  }
}
```

UNIT1 Fundamental Programming in Java

The Java Programming Environment- JVM, JIT Compiler, A Simple Java Program, Data Types, Variables, Operators, Strings, Input and Output, Control, Flow Declaring Classes, Declaring Member Variables, Defining Methods, Constructor, Passing Information to a Method or a Constructor, Creating and using objects, Controlling Access to Class Members, Static Fields and Methods, this keyword

1. Explain Procedure Programming language with example of C language. What are advantages and disadvantages of C Programming language

Procedure Oriented Programming (POP) is a programming approach where the program is divided into small functions or procedures. Each function performs a specific task and the program executes step by step. The main focus is on functions and the logic of the program rather than data. In POP, data is shared among functions and the program is designed using a top-down approach. Languages like C, Pascal and Fortran follow this programming style.
Example of C Program

```
#include<stdio.h>

void display(){
  printf("Welcome to C Programming");
}

int main(){
  display();
  return 0;
}
```

Advantages of C Programming Language: Simple and easy to learn for beginners, supports structured programming using functions, programs are very fast and efficient, portable language that can run on different machines, provides rich set of library functions, widely used in system programming such as operating systems and embedded systems.
Disadvantages of C Programming Language: Does not support object oriented concepts like inheritance and polymorphism, no automatic memory management, low data security because data can be accessed globally, pointer concept is complex for beginners, no built in exception handling.

2. What is Python? Explain its advantages and disadvantages. Write a simple Python program to implement a Student Result System that calculates total and average marks of a student

Python is a high level, interpreted and general purpose programming language created by Guido van Rossum in 1991. It is known for its simple syntax and readability which makes it easy to learn and use. Python supports multiple programming paradigms such as procedural, object oriented and functional programming. It is widely used in web development, data science, artificial intelligence, machine learning and automation.
Advantages of Python: Easy to learn and simple syntax, fewer lines of code compared to other languages, large standard library and frameworks available, platform independent and runs on different operating systems, open source and free to use, supports rapid application development.
Disadvantages of Python: Slower execution speed compared to compiled languages

like C or C++, high memory consumption, not widely used for mobile application development, weaker database access compared to some enterprise languages, sometimes not suitable for high performance applications.
Python Program for Student Result System

```
name = input("Enter student name: ")
m1 = int(input("Enter marks of subject 1: "))
m2 = int(input("Enter marks of subject 2: "))
m3 = int(input("Enter marks of subject 3: "))
total = m1 + m2 + m3
average = total / 3
print("Student Name:", name)
print("Total Marks:", total)
print("Average Marks:", average)
```

3. Explain the concept of Object-Oriented Programming (OOP). Discuss the main principles of OOP and describe the important features of Java

Object Oriented Programming (OOP) is a programming paradigm that is based on the concept of objects. Objects represent real world entities and contain both data and methods. In OOP, programs are organized using classes and objects which makes the software modular, reusable and easier to maintain. A class acts as a blueprint and objects are instances of that class.

Main Principles of OOP: Encapsulation – Encapsulation means wrapping data and methods together into a single unit called a class and restricting direct access to data to improve security. Inheritance – Inheritance allows one class to acquire the properties and methods of another class which promotes code reuse and hierarchical classification. Polymorphism – Polymorphism means one interface can perform multiple tasks. It allows the same method name to behave differently depending on the situation through method overloading or method overriding. Abstraction – Abstraction means hiding internal implementation details and showing only the essential features of an object to the user which reduces program complexity.

Important Features of Java: Simple language with easy syntax similar to C and C++, object oriented language that supports class and object concepts, platform independent because Java programs run on the Java Virtual Machine allowing Write Once Run Anywhere concept, secure language with features like bytecode verification and no pointer access, robust with strong memory management and exception handling, multithreaded which allows multiple tasks to run simultaneously, portable and architecture neutral meaning programs can run on different systems, high performance with Just In Time compiler, distributed which supports network based

applications, dynamic and extensible allowing new classes and libraries to be added easily.

4. Explain JDK, JRE and JVM with the block Diagram. List the steps and command used to compile and run Java source Code

In Java, programs are developed and executed using three important components: JDK, JRE and JVM. These components work together to compile and run Java programs on different platforms.

JDK (Java Development Kit): JDK is a complete software development kit used to develop Java applications. It contains tools required to write, compile and run Java programs. It includes the Java compiler (javac), debugger, documentation generator and other development tools. JDK also contains JRE which provides the runtime environment.

JRE (Java Runtime Environment): JRE provides the environment required to run Java programs. It includes JVM and standard class libraries needed for program execution. JRE does not contain development tools; it is only used to run compiled Java programs.

JVM (Java Virtual Machine): JVM is a virtual machine that executes Java bytecode. It converts the bytecode into machine code so that the computer can understand and run it. JVM is responsible for memory management, security and making Java platform independent.

Block Diagram

```
Java Source Code (.java)
      |
      v
   JDK (Compiler - javac)
      |
      v
  Bytecode (.class)
      |
      v
      JRE
  -----------
  |    JVM    |
  | Class Loader |
  | Memory Mgmt |
  -----------
      |
      v
  Machine Code
```

|
v
Output

Steps to Compile and Run Java Program: Write the Java program and save the file with .java extension. Compile the program using the Java compiler. The compiler converts source code into bytecode. Run the compiled program using JVM. JVM interprets the bytecode and produces output.

Commands Used:
Compile Command

javac Hello.java

Run Command

java Hello

5. Explain Java Virtual Machine and its various component with neat diagram
The Java Virtual Machine (JVM) is the part of Java that executes compiled Java bytecode. It acts as an interpreter between the Java program and the computer hardware. JVM allows Java programs to run on different platforms without modification which supports the concept of "Write Once Run Anywhere". JVM loads the bytecode, verifies it, executes it and manages memory during program execution.

Main Components of JVM:
Class Loader: Loads compiled Java classes into memory. It reads the .class file and loads it into JVM for execution.
Method Area: Stores class structures such as metadata, constant pool, method data and field information.
Heap Memory: Stores objects created during program execution. All objects and instance variables are stored here.
Stack Memory: Each thread has its own stack used for method calls, local variables and partial results.
Program Counter Register: Keeps track of the current instruction being executed by the JVM.
Native Method Stack: Used for executing native methods written in languages like C or C++.
Execution Engine: Executes the bytecode instructions. It includes the interpreter and Just In Time (JIT) compiler.
Garbage Collector: Automatically removes unused objects from memory and frees space.

JVM Architecture Diagram

```
 Java Program
     |
     v
 Class Loader
     |
     v
 -----------
 | Method Area |
 ---------------
     |
---------------
|    Heap     |
---------------
     |
---------------
|    Stack    |
---------------
     |
---------------
| Execution Engine |
| Interpreter + JIT |
---------------
     |
     v
  Output
```

6. Describe Primitive Datatypes available in Java with help of purpose, Syntax and Example

Primitive data types are the basic built-in data types provided by Java to store simple values such as numbers, characters and logical values. They are not objects and directly store values in memory. Java provides eight primitive data types.

byte
Purpose: Used to store small integer values and save memory. Range −128 to 127.
Syntax: byte byteVar;
Example: byte age = 20;

short
Purpose: Used to store integer numbers with a larger range than byte.

Syntax: short shortVar;
Example: short marks = 300;

int
Purpose: Used to store integer numbers. It is the most commonly used numeric data type.
Syntax: int intVar;
Example: int salary = 50000;

long
Purpose: Used to store very large integer values.
Syntax: long longVar;
Example: long population = 9000000000L;

float
Purpose: Used to store decimal numbers with single precision.
Syntax: float floatVar;
Example: float price = 45.75f;

double
Purpose: Used to store large decimal numbers with double precision.
Syntax: double doubleVar;
Example: double percentage = 89.65;

char
Purpose: Used to store a single character such as letters or symbols.
Syntax: char charVar;
Example: char grade = 'A';

boolean
Purpose: Used to store logical values true or false.
Syntax: boolean booleanVar;
Example: boolean isButtonOn = true;

Summary Java Program Showing All Primitive Data Types

```
class DataTypesDemo {
  public static void main(String args[]) {

    byte b = 10;
    short s = 200;
    int i = 5000;
    long l = 9000000000L;

    float f = 5.5f;
```

```
    double d = 45.678;

    char c = 'A';
    boolean flag = true;

    System.out.println("byte value: " + b);
    System.out.println("short value: " + s);
    System.out.println("int value: " + i);
    System.out.println("long value: " + l);
    System.out.println("float value: " + f);
    System.out.println("double value: " + d);
    System.out.println("char value: " + c);
    System.out.println("boolean value: " + flag);
  }
}
```

7. Describe Arithmetic and Assignment operators with help of Symbol, Name, Description and apply the operators on x and y variable for the example demonstration

In Java, operators are special symbols used to perform operations on variables and values. Arithmetic operators perform mathematical calculations, while assignment operators assign values to variables and update them. Assume the variables x = 10 and y = 5 for demonstration.

Arithmetic Operators
Symbol: +
Name: Addition
Description: Adds two values.
Example: x + y = 10 + 5 = 15

Symbol: -
Name: Subtraction
Description: Subtracts the second value from the first value.
Example: x - y = 10 - 5 = 5

Symbol: *
Name: Multiplication
Description: Multiplies two values.
Example: x * y = 10 × 5 = 50

Symbol: /
Name: Division

Description: Divides the first value by the second value.
Example: x / y = 10 / 5 = 2

Symbol: %
Name: Modulus
Description: Returns the remainder of division.
Example: x % y = 10 % 5 = 0

Assignment Operators
Symbol: =
Name: Simple Assignment
Description: Assigns a value to a variable.
Example: x = y means x becomes 5

Symbol: +=
Name: Add and Assign
Description: Adds value and stores the result in the same variable.
Example: x += y → x = x + y = 15

Symbol: -=
Name: Subtract and Assign
Description: Subtracts value and stores the result.
Example: x -= y → x = 5

Symbol: *=
Name: Multiply and Assign
Description: Multiplies value and stores the result.
Example: x *= y → x = 50

Symbol: /=
Name: Divide and Assign
Description: Divides value and stores the result.
Example: x /= y → x = 2

Symbol: %=
Name: Modulus and Assign
Description: Finds remainder and stores the result.
Example: x %= y → x = 0

Summary Java Program Showing All Operators

```
class OperatorsDemo {
  public static void main(String args[]) {

    int x = 10;
    int y = 5;
```

```
        System.out.println("Addition: " + (x + y));
        System.out.println("Subtraction: " + (x - y));
        System.out.println("Multiplication: " + (x * y));
        System.out.println("Division: " + (x / y));
        System.out.println("Modulus: " + (x % y));

        x += y;
        System.out.println("x += y: " + x);

        x -= y;
        System.out.println("x -= y: " + x);

        x *= y;
        System.out.println("x *= y: " + x);

        x /= y;
        System.out.println("x /= y: " + x);

        x %= y;
        System.out.println("x %= y: " + x);
    }
}
```

8. Explain the Java Comments, Source File, Class, Method Declaration rules with Sample Java Source Code

Java programs follow specific rules related to comments, source files, classes and methods which help in writing structured and understandable code.

Java Comments: Comments are used to explain the code and improve readability. They are ignored by the compiler and do not affect program execution. Types of comments include single line comments using //, multi line comments using /* */, and documentation comments using /** */ which are used to generate documentation using Javadoc.

Java Source File Rules: Java program files must have the extension .java. If a class is declared public, the file name must be the same as the class name. A single source file can contain multiple classes but only one class can be public. The program execution starts from the main() method.

Class Declaration Rules: A class is declared using the keyword class. The class name usually starts with a capital letter by convention. The class body is written inside curly braces { } and contains variables and methods. A class acts as a blueprint for creating objects.

Method Declaration Rules: A method is a block of code that performs a specific task. A method must have a return type, method name and parentheses. Parameters can be passed inside parentheses if required. The body of the method is written inside curly braces. The main() method is the starting point of a Java program.

Sample Java Source Code

```
// Example Java Program

class HelloWorld {

  /* main method is the entry point
    of the Java program */

  public static void main(String args[]) {

    System.out.println("Hello Java Programming");
  }
}
```

Pro Interview Question: How many ways Java object can be created?
Using new keyword, using reflection, using clone() method, using deserialization.

G. What is Scanner Class? Explain various methods available in Scanner class. Write a sample program to take String, integer, Float and double variables from command prompt

The Scanner class is used in Java to read input from the user through the keyboard. It is available in the java.util package. Scanner allows programs to accept different types of input such as integers, floating point numbers and strings. To use the Scanner class, it must be imported using import java.util.Scanner;. An object of the Scanner class is then created to read input from the keyboard using System.in.

Common Methods of Scanner Class

nextInt() – reads an integer value from input.
nextFloat() – reads a float value from input.
nextDouble() – reads a double value from input.
next() – reads a single word string.
nextLine() – reads an entire line of text including spaces.
nextBoolean() – reads a boolean value true or false.

Sample Program

```
import java.util.Scanner;

class ScannerExample {
```

```
    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Age: ");
        int age = sc.nextInt();

        System.out.print("Enter Height (float): ");
        float height = sc.nextFloat();

        System.out.print("Enter Percentage (double): ");
        double percentage = sc.nextDouble();

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Height: " + height);
        System.out.println("Percentage: " + percentage);
    }
}
```

10. What is Class and Object in Java? Explain Data members and Methods of Class with the help of diagram and example

In Java, a **class** is a blueprint or template used to create objects. It defines the properties (variables) and behaviors (methods) that the objects will have. A class does not occupy memory until an object of that class is created. An **object** is an instance of a class. It represents a real world entity and contains actual values for the properties defined in the class. Objects use the variables and methods defined in the class to perform operations.

Data Members: Data members are variables declared inside a class that store data or attributes of an object. They represent the properties of the object such as name, id, marks, etc.

Methods: Methods are functions defined inside a class that perform operations on the data members. They define the behavior of objects. Methods help in organizing code and allow reuse of functionality.

Diagram

```
      Class : Student
    ----------------
    | Data Members       |
    | id                 |
    | name               |
    | marks              |
    |                    |
    | Methods            |
    | display()          |
    | calculateResult()  |
    ----------------
          |
          | create object
          v
       Object : s1
```

Example Java Program

```
class Student {

  int id;
  String name;

  void display() {
    System.out.println("ID: " + id);
    System.out.println("Name: " + name);
  }

  public static void main(String args[]) {

    Student s1 = new Student();

    s1.id = 101;
    s1.name = "Rahul";

    s1.display();
  }
}
```

11. **Write a short note**

A. Operators – new operator, instanceof operator, dot(.) operator
new Operator: The new operator is used to create objects of a class. It allocates

memory for the object and calls the constructor of the class. Syntax: ClassName objectName = new ClassName(); Example: Student s1 = new Student();
instanceof Operator: The instanceof operator is used to check whether an object belongs to a particular class or not. It returns true or false. Syntax: objectName instanceof ClassName Example:

```
String str = "Java";
System.out.println(str instanceof String);
```

dot (.) Operator: The dot operator is used to access data members and methods of a class using an object. Syntax: objectName.variableName or objectName.methodName() Example:

```
Student s1 = new Student();
s1.id = 10;
s1.display();
```

B. If, else, if-else with syntax and example
If Statement: The if statement is used to execute a block of code when a condition is true. Syntax:

```
if(condition){
  // statements
}
```

Example:

```
int marks = 60;
if(marks >= 50){
  System.out.println("Pass");
}
```

If-Else Statement: The if-else statement executes one block of code if the condition is true and another block if the condition is false. Syntax:

```
if(condition){
  // true statements
}
else{
  // false statements
}
```

Example:

```
int number = 7;
if(number % 2 == 0){
  System.out.println("Even Number");
```

```
}
else{
   System.out.println("Odd Number");
}
```

12. Design flow charts for loops and conditions

1. Flowchart for For Loop

```
Start
 |
Initialize (i=1)
 |
Check Condition (i <= n)
 |Yes
 v
Execute Statements
 |
Increment (i++)
 |
Go back to Condition
 |
No
v
End
```

2. Flowchart for While Loop

```
Start
 |
Initialize variable
 |
Check Condition
 |Yes
 v
Execute Statements
 |
Update Variable
 |
Go back to Condition
 |
 No
```

```
 v
End
```

3. Flowchart for Do-While Loop

```
Start
 |
Execute Statements
 |
Update Variable
 |
Check Condition
 |Yes -----
 |      |
 -------
    |
   No
    |
   End
```

4. Flowchart for If-Else Condition

```
Start
 |
Check Condition
 |True     |False
 v         v
Execute IF  Execute ELSE
Statements  Statements
   \      /
    \    /
     End
```

5. Flowchart for Switch Statement

```
Start
 |
Read Expression
 |
Check Case 1 ? → Execute Case 1
 |
Check Case 2 ? → Execute Case 2
 |
Check Case 3 ? → Execute Case 3
```

|

Default Case → Execute Default

|

End